

Nama : Imron
NPM : 232310021
Mata Kuliah : LAB PW TI24PA2

Latihan 1

forms.jsx berfungsi sebagai penyedia komponen input global yang reusable dan modular untuk mendukung pembuatan formulir pengisian data buku pada modul CMS. Di dalam file ini, dibuat beberapa komponen cetakan input seperti TextInput untuk input teks biasa, TextAreaInput untuk narasi panjang seperti sinopsis dan cerita, InputCheckbox untuk pilihan tipe buku, serta InputImage yang dilengkapi dengan pratinjau gambar untuk fitur unggah sampul buku. Setiap komponen input secara otomatis dibungkus dengan label dinamis lewat komponen LableTitle untuk menjaga konsistensi visual antarmuka. Selain komponen visual, berkas ini juga mengimplementasikan fungsi keamanan data di sisi klien berupa validateInput berbasis ekspresi reguler untuk menyaring karakter yang diizinkan, serta fungsi sanitizeInput untuk membersihkan atau menghapus karakter berbahaya dari baris input sebelum diproses lebih lanjut.

```

forms.jsx

const ALLOWED_PATTERN =
  /^[a-zA-Z0-9\s.,!?@#%&*()\-_+=;:''>\[\]\{\}\|\~`]*$/;

const validateInput = (value) => {
  return ALLOWED_PATTERN.test(value);
};

const sanitizeInput = (value) => {
  return value.replace(
    /[\^a-zA-Z0-9\s.,!?@#%&*()\-_+=;:''>\[\]\{\}\|\~`]/g,
    ""
  );
};

const LabelTitle = ({ title, required }) => {
  return (
    <label className={`form-label fw-semibold ${required ? "required" : ""}`}>
      {title}
    </label>
  );
};

const TextInput = ({ title, required, ...props }) => {
  return (
    <div className="form-group mb-3">
      {title} && <LabelTitle title={title} required={required} />
      <input type="text" required={required} className="form-control" {...props} />
    </div>
  );
};

const TextAreaInput = ({ title, required, ...props }) => {
  return (
    <div className="form-group mb-3">
      {title} && <LabelTitle title={title} required={required} />
      <textarea required={required} className="form-control" {...props}></textarea>
    </div>
  );
};

const InputCheckbox = ({
  title,
  value,
  required,
  is_switch = false,
  ...props
}) => {
  return (
    <div className="form-group">
      {title} && <LabelTitle title={title} required={required} />
      <div className={`form-check ${is_switch ? "form-switch" : ""}`}>
        <input
          type="checkbox"
          className="form-check-input"
          required={required}
          {...props}
        />
        <label className="form-check-label ms-1">{value}</label>
      </div>
    </div>
  );
};

const InputImage = ({ title, imagePreview, required, ...props }) => {
  return (
    <div className="form-group mb-3">
      {title} && <LabelTitle title={title} required={required} />
      <input
        type="file"
        className="form-control"
        id="coverImage"
        name="coverImage"
        accept="image/*"
        required={required}
        {...props}
      />
      {imagePreview} && (
        <div className="mt-3">
          <img
            src={imagePreview}
            alt="Cover Preview"
            style={{
              maxWidth: "200px",
              maxHeight: "300px",
              objectFit: "cover",
            }}
            className="img-thumbnail"
          />
        </div>
      )
    </div>
  );
};

export {
  validateInput,
  sanitizeInput,
  TextInput,
  TextAreaInput,
  InputCheckbox,
  InputImage,
};

```

Latihan 2

Proses penambahan data buku baru diintegrasikan melalui fungsi `handleAdd` yang dipicu ketika pengguna menekan tombol "Add New Book" pada komponen header manajemen buku. Fungsi ini memanfaatkan utilitas global `openModal` untuk memunculkan jendela pop-up interaktif berukuran besar di atas layar utama tanpa mengganggu alur kerja pengguna. Di dalam jendela modal tersebut, komponen formulir utama dipanggil dan disuntikkan sebuah properti fungsi callback bernama `onSubmit`. Ketika pengguna selesai mengisi seluruh field dan mengirimkan data dari formulir, fungsi callback tersebut secara otomatis menangkap objek data buku baru dan menggabungkannya ke dalam state daftar buku utama menggunakan metode pembaruan asinkron, sehingga koleksi buku pada tabel manajemen langsung terbaru secara real-time.

```
index.jsx

const handleAdd = () => {
  openModal({
    open: true,
    header: "",
    size: "lg",
    message: (
      <Form onSubmit={(newBook) => setBooks((prev) => [newBook, ...prev])} />
    ),
  });
};
```

Output

The screenshot displays the 'Books Management' application interface. The top section shows the 'Add New Book' modal, which is a form for adding a new book. The modal includes fields for 'Book Title', 'Author Name', 'Synopsis', 'Story', and 'Cover Image'. There is also a 'Type Book' toggle switch set to 'Is Free'. The 'Submit' button is highlighted in blue. Below the modal, the 'Book Lists' table is visible, showing a list of books with columns for 'NO', 'TITLE', 'AUTHOR', 'LANGUAGE', 'RATE/VIEW', 'SUBSCRIBE', and 'ACTIONS'. The table contains 6 rows of data, including books like 'TEST TITLE', 'Harry Potter and the Sorcerer's Stone', 'Hunger Games', 'Maze Runner', and 'Divergent'.

NO	TITLE	AUTHOR	LANGUAGE	RATE/VIEW	SUBSCRIBE	ACTIONS
1	TEST TITLE	TEST AUTHOR	English	★ 0 0	Yes	[Edit] [Delete]
2	jiji	k	English	★ 0 0	Yes	[Edit] [Delete]
3	Harry Potter and the Sorcerer's Stone	J.K. Rowling	English	★ 4.5 1000	No	[Edit] [Delete]
4	Hunger Games	Suzanne Collins	English	★ 4.9 2000	No	[Edit] [Delete]
5	Maze Runner	James Dashner	English	★ 4.7 500	No	[Edit] [Delete]
6	Divergent	Veronica Roth	English	★ 3.7 300	No	[Edit] [Delete]